

Appendix M — Concurrency and Rate-Limiting Patterns Cookbook

Every pattern in this appendix guards the same chokepoint: `llm_invoke()`, the one function every LLM call in either pipeline passes through. Two of the five patterns named here are not the code shipping today — they are the project's own earlier, abandoned designs, recovered from the raw chat transcripts that wrote them, since neither survives in any current `.py` file. That history is kept here on purpose: it explains why the current code looks the way it does, which a snapshot of the final version alone cannot.

M.1 ThreadPoolExecutor timeout wrapper

`signal.alarm()` was the obvious first choice for enforcing a call timeout and was rejected for two concrete reasons: it does not exist on Windows, the project's development platform, and it cannot interrupt a thread that is inside a C extension holding the GIL — exactly the situation for every call this pattern wraps (NumPy inside SentenceTransformers, ChromaDB's native query layer, an HTTP client waiting on a socket). `ThreadPoolExecutor(max_workers=1)` plus `future.result(timeout=N)` works because it does not require cooperation from the wrapped call at all:

```
with concurrent.futures.ThreadPoolExecutor(max_workers=1) as _executor:
    _future = _executor.submit(llm.invoke, messages, config=config,
                               **kwargs)
    try:
        response = _future.result(timeout=LLM_RESPONSE_TIMEOUT_SECONDS)
    except concurrent.futures.TimeoutError:
        raise LLMResponseTimeoutError(LLM_RESPONSE_TIMEOUT_SECONDS)
```

The same shape wraps two other blocking calls the project depends on — `collection.query()` in `retriever.py` (timeout swallowed, returns `[]` so the pipeline continues without that collection) and `model.encode()` in `embedding_manager.py` (timeout re-raised, since no meaningful fallback embedding exists) — confirming this is a house pattern, not an LLM-specific one. Thresholds were calibrated to roughly 2× the observed p95 latency for LLM calls and far wider margins for retrieval and embedding, whose latency variance is much lower.

Common pitfall — The timeout unblocks the caller, not the work

Only the calling thread stops waiting at the deadline — the background worker thread is not killed and keeps running to completion, still holding whatever resources it acquired. This is a deliberate, accepted tradeoff here, not an oversight: none of the three wrapped calls can hang forever (the LLM provider enforces its own timeout, ChromaDB is local, SentenceTransformers has no network I/O), so the leaked thread is always bounded. A future call site wrapped in this same pattern needs the same guarantee, or the leak stops being bounded.

M.2 Semaphore-guarded LLM calls — tried, then abandoned

For part of one working session, `'app/llm_caller.py'` gated every LLM call behind a `'threading.Semaphore(1)'` rather than the FIFO queue described in §M.3. `'threading.Semaphore'` was chosen over `'asyncio.Semaphore'` for a specific, structural reason: LLM calls run inside `'asyncio.to_thread()'`, executing in worker threads outside the event loop, and an `'asyncio.Semaphore'` can only be awaited — it cannot be acquired from a blocking thread.

```
_LLM_MAX_CONCURRENT: int = 1
_llm_semaphore = threading.Semaphore(_LLM_MAX_CONCURRENT)

_llm_semaphore.acquire()
try:
    result = _invoke_once(...)
finally:
    _llm_semaphore.release()    # released here, before the backoff sleep

time.sleep(delay)             # sleep happens outside the semaphore
```

Releasing before the sleep, rather than after, was itself a deliberate choice — holding the semaphore through a multi-second backoff would stall every other waiting thread for that thread's entire cooldown. But this design has a gap a semaphore cannot close: it grants **admission**, not **ordering**. A thread that just received a 429 releases the semaphore before it finishes sleeping out the rate-limit window, and the instant it releases, any newly arrived thread can re-acquire it and immediately burn the same still-exhausted token window — the 429-holder has no way to say “I go next, even while I'm asleep.” This was discovered live, mid-session: a `'pre_attempt_delay'` patch (sleeping before re-acquiring rather than after releasing) was tried as an intermediate fix, but the team concluded a semaphore fundamentally cannot express reserved ordering — which is exactly what a FIFO queue provides for free.

M.3 FIFO-queue serializer

The design that replaced the semaphore, later that same session, and the one shipping in both pipelines today. `_gate_acquire()` takes the gate immediately if nothing else is active; otherwise it enqueues a per-thread `threading.Event` and blocks on it — new arrivals always go to the back of the line.

```
_llm_queue: queue.Queue[threading.Event] = queue.Queue()
_llm_gate_lock: threading.Lock = threading.Lock()
_llm_active: bool = False

def _gate_acquire(caller_tag: str) -> None:
    global _llm_active
    with _llm_gate_lock:
        if not _llm_active:
            _llm_active = True
            return
        my_event = threading.Event()
        _llm_queue.put(my_event)
        my_event.wait()

def _gate_release_to_next() -> None:
    global _llm_active
    with _llm_gate_lock:
        try:
            next_event = _llm_queue.get_nowait()
            next_event.set()          # _llm_active stays True — handed over
        except queue.Empty:
            _llm_active = False      # nobody waiting; gate is free
```

The property the semaphore design could not express is stated directly in the shipped module's own docstring: a thread that gets a 429 does not re-enqueue — it holds the gate, sleeps the full reset window, then retries directly, with the gate released to the next waiter only after a terminal success or a non-retryable error. This closes the exact race the semaphore design hit: nothing can cut in front of a thread that is mid-backoff, because it never gave up its place in line to begin with.

The bug that forced this rewrite is worth stating precisely, since it is the reason this pattern exists at all: two simultaneous `POST /query` requests, dispatched via `asyncio.to_thread()`, looked serialized by an `async with _query_lock:` in the API handler — but an `asyncio.Lock` releases the instant `to_thread()` hands work to a worker thread, so it is not a cross-thread mutex at all. The first request's calls exhausted the provider's token-per-minute quota; the second request hit 429 on every retry, exhausted its attempt budget, and returned a failed result with no exception raised at the HTTP boundary — a silently bad answer, not a visible error. Moving serialization down to `llm_invoke()` itself, the one chokepoint every call actually passes through, is what closed the gap; the now-redundant `async with _query_lock:` was removed from both API handlers entirely.

M.4 Capped exponential backoff with jitter — and why the jitter left

Before the FIFO gate existed, backoff between retries used the conventional shape: an exponential delay capped at a hard ceiling, plus a small random jitter term to keep multiple racing threads from retrying in lockstep.

```
exponential = min(max_seconds, base_seconds * (2 ** (attempt - 1)))
delay += random.uniform(0.0, jitter_seconds)
```

With the project's real constants — `base\_seconds=1.0`, `max\_seconds=30.0` — the complete exponential sequence by attempt number is 1s, 2s, 4s, 8s, 16s, then capped flat at 30s from the sixth attempt onward ($2^5=32$ exceeds the 30s ceiling), plus 0–0.25s of jitter on top of whichever value applies. Once the FIFO gate (§M.3) made strict single-caller ordering a guarantee rather than a hope, jitter's entire reason for ordering a guarantee rather than a hope, jitter's entire reason for existing disappeared — the shipped code's own docstring says so directly:

```
def _rate_limit_delay(
    result: LLMResult,
    attempt: int,
    *,
    base_seconds: float,
    max_seconds: float,
) -> float:
    """
    Compute how long to wait before the next retry attempt and publish the
    reset deadline globally so _wait_for_token_window() can use it.

    Jitter is intentionally absent: with the FIFO gate only one thread
    calls
    Groq at a time, so there is no collision to de-synchronize.
    """
    exponential = min(max_seconds, base_seconds * (2 ** (attempt - 1)))
    header_hint = _groq_wait_seconds(result)
    delay = max(exponential, header_hint)
    _update_token_reset(delay)
    return delay
```

Common pitfall — A config constant that outlived the code reading it

`LLM\_RATE\_LIMIT\_BACKOFF\_JITTER\_SECONDS` is still defined in `config.py` and is not imported anywhere in the current call chain — confirmed by grep, and called out directly in the project's own commit history as deliberate: the constant was left in place specifically because deleting a config value is a larger, riskier edit than simply no longer reading it. A stale-looking constant is not automatically a bug; check whether the code that consumed it was removed on purpose before deleting it.

M.5 LLMRateLimitAbortError handling

The FIFO gate and jitter-free backoff still leave one open question: what happens when the required backoff itself becomes unreasonable — a multi-minute sleep that, in a request-handling context, would tie up a server thread indefinitely with no signal to the client. `LLMRateLimitAbortError` is the closed answer to that question, raised inside the retry loop before the delay is ever slept:

```
class LLMRateLimitAbortError(Exception):
    def __init__(self, delay: float) -> None:
        self.delay = delay
        super().__init__(f"Rate-limit backoff of {delay:.0f}s exceeds maximum allowed delay.")

    if delay >= rate_limit_max_delay_seconds:
        _gate_release_to_next() # release before raising — never strand a waiter
        raise LLMRateLimitAbortError(delay)
```

Releasing the gate before raising is not incidental — raising first would strand every other queued caller behind a gate nobody would ever release. `LLM_RATE_LIMIT_MAX_DELAY_SECONDS = 1800.0` (30 minutes) is deliberately generous: a required delay of that magnitude almost certainly signals a genuine quota suspension rather than an ordinary transient burst, at which point sleeping through it would simply waste time better spent surfacing the failure. Four real call sites catch it, each translating the same `'delay'` field into the vocabulary its caller understands:

Caller	Handling
<code>`app/agent_query.py` (CLI)</code>	Prints the required wait in seconds, continues the input loop
<code>`app_workflow/main.py` (CLI)</code>	Formats the delay as minutes and seconds before printing
<code>`app/api.py` / `app_workflow/api.py` (HTTP)</code>	<code>'HTTPException(429, ...)'</code> with a <code>'Retry-After'</code> header set to the delay

This exception is only reachable because ``_invoke_once()``'s exception-handling chain is wide enough to catch a 429 regardless of which layer it surfaces through — a provider SDK's own typed rate-limit error, a raw ``httpx.HTTPStatusError`` once an SDK's internal retry logic has already given up, or a raw ``requests``-level error. A rate-limit abort mechanism is only as reliable as the exception taxonomy feeding it; the two are the same cookbook entry in practice, even though only one of them has a name.